

MODULE 6 ADD-ON · CLASSROOM EXERCISE

Ship a Feature with an AI Pair

Fast, but owned — practising the AI playbook on a real banking story

30 minutes total · 20 work + 5 present + 5 Q&A · Teams of 3–4

Exercise at a glance

Print or project this slide while teams work

20 min

Team work

5 min

One team presents

5 min

Q&A + trainer debrief

Setup

- Teams of 3–4; one laptop/notepad per team
- Hand out (or project) the scenario on slide 3
- No AI tool access needed — teams simulate the prompt and predict the AI's output on paper
- This directly practises Module 6's AI playbook (habits 1, 3, 6) and slide 27's guard-rails

Deliverable per team

- 1. User story + 3 acceptance criteria
- 2. The prompt they'd give an AI coding assistant, plus the plan they predict it returns
- 3. A 3-item diff-review checklist
- 4. One judgement call AI cannot make for them

The scenario



Feature request: "Freeze My Card"

Customers suspect fraud on their debit card but the branch is closed and the call centre queue is 20 minutes. Add a button in the banking app so a customer can instantly freeze — and later unfreeze — their own card, without calling support.

Your task, in your team (20 minutes)

- Step 1 (5 min): Write the user story and 3 testable Given/When/Then acceptance criteria (see slide 10, Module 2 style)
- Step 2 (5 min): Draft the prompt you'd give an AI coding assistant (story + AC + test command); predict its plan
- Step 3 (5 min): List 3 things a senior would check in the AI's diff before merging (slide 27's review habit)
- Step 4 (5 min): Name one decision on this feature that AI cannot make for you, and why

Facilitation tips

While teams work (20 min)

- Call out the 5-minute mark for each of the 4 steps — most teams rush step 1 and run out of time for step 4
- If a team's acceptance criteria have no number or Given/When/Then shape, point them back to slide 10
- Nudge teams stuck on step 4 with: 'What would a bank's risk or compliance team ask before launch?'
- Walk the room — this is where you spot the misconception to correct in Q&A

Common mistakes to watch for

- Acceptance criteria that describe the UI ('button should be blue') instead of behaviour
- A 'plan' that is really just the finished code — teams skip predicting what the AI proposes before code
- A diff-review list that only mentions 'check it works' — not naming, error paths, tests, secrets
- Step 4 answers that are actually engineering questions (e.g. 'which database') rather than judgement calls

Presentation & Q&A (10 minutes)

Presentation (5 min)

- One team presents all 4 deliverables in under 4 minutes
- Ask them to read their step-4 judgement call last — it usually sparks the best discussion

Q&A (5 min)

- Ask the room: did anyone's step-4 answer differ?
- Reveal the model solution (next slides) and compare

Debrief line to say out loud

"Notice that steps 1–3 are things AI can help draft quickly. Step 4 is the one no prompt can answer for you — that's the part of the job that doesn't go away."



MODEL SOLUTION

Trainer's step-by-step answer

Reveal after teams present — use it to compare, not to grade harshly

Step 1 — User story & acceptance criteria

As a **customer**, I want to **freeze my debit card instantly from the app** so that **I can stop misuse without waiting on hold**.

✓ **Given** my card is active **when** I tap Freeze **then** transactions are declined within 5 seconds and I see a confirmation

✓ **Given** my card is frozen **when** I tap Unfreeze **then** the card becomes usable again within 5 seconds

✓ **Given** my card is frozen **when** a merchant attempts a transaction **then** it is declined with reason 'card frozen by customer', visible to support

Notice: each line has a number or a named, checkable outcome — none is a wish.

Step 2 — The prompt, and the plan you'd expect back



Prompt to the AI coding assistant:

"Implement this story for our banking app: [story + 3 acceptance criteria above]. Before writing code, give me a plan: which files/services you'll touch, the API endpoint you'll add, the test cases you'll write, and any risks you see. Test command: `npm test freeze-card.spec.ts`"

< > Plan you should expect the AI to propose

- New endpoint `POST /cards/{id}/freeze` and `POST /cards/{id}/unfreeze`, updating a `card-status` field
- An event emitted on freeze/unfreeze so downstream services (statements, fraud monitoring) stay in sync
- Tests for: happy path freeze, happy path unfreeze, freezing an already-frozen card, freezing a closed/invalid card
- A flagged risk: what should happen to a transaction already in flight when the freeze request arrives?

Step 3 — Reviewing the AI's diff like a senior would

Following slide 27: naming, error paths, tests, secrets

Naming

Does the code call it 'freeze', 'lock' or 'block'? It must match the term already used elsewhere in the banking domain, or support tickets and logs will disagree with the UI.

Error paths

What does the API return if the card is already frozen, already closed, or reported lost? A missing case here becomes a confusing error message in production.

Tests

Do the generated tests cover the race condition of freeze and unfreeze arriving at nearly the same time, and the merchant-decline path from AC 3 — not just the two happy paths?

Secrets

No real or example card numbers (PANs), tokens, or customer IDs anywhere in the code or test fixtures — use clearly fake placeholder data only.

Step 4 — The judgement call AI cannot make

This is the part of slide 27's 'exactly as hard' box in action

The question

Should a frozen card still let a pending pre-authorised payment (e.g. a subscription already approved) go through, or should absolutely everything be blocked the instant the customer taps Freeze?

Why AI cannot answer this for you

- It is a risk-policy and regulatory decision, not a coding decision — the 'right' answer depends on the bank's fraud tolerance, card-network rules, and customer-experience commitments
- It also needs sign-off from risk, compliance and customer support — not from a code reviewer
- The AI can implement whichever rule the team decides on quickly, but it cannot decide the rule — that ownership stays with the product engineer and the business, per Module 1 and slide 27

Debrief: what this exercise showed

- 1 Steps 1–3 (story, prompt, diff review) are exactly where an AI assistant saves real time
- 2 Step 4 never gets faster with a better prompt — it needs business, risk and regulatory judgement
- 3 The habit that mattered most today: 'Ask for a plan before code' and 'Use it as a critic', from the Module 6 playbook
- 4 Carry this pattern into your own project work: spec first, plan before code, review like a senior, keep the judgement calls